

STUDENT PERFORMANCE MANAGEMENT SYSTEM

A Java Console Application for Student Records, Marks & Performance Analytics

VITyarthi — Build Your Own Project

Submitted by: Piyush

Registration No.: 25BAI11043

Program: B.Tech, Computer Science & Engineering

Institution: VIT Bhopal University

1. Introduction

Every classroom generates a steady stream of numbers - marks, percentages, grades, class averages - that someone has to track and interpret. Doing this by hand or in an ad-hoc spreadsheet is slow and error-prone once the class grows beyond a handful of students. This project builds a self-contained Java console application that automates that entire workflow: maintaining a student roster, recording subject-wise marks, and instantly generating performance reports.

The project was built as a VITYarthi 'Build Your Own Project' submission, applying core Java concepts - object-oriented design, the Collections Framework, file I/O, and exception handling - to a realistic, self-contained problem.

2. Problem Statement

Manually tracking student marks, percentages, and grades is error-prone and time-consuming when done with pen-and-paper or plain spreadsheets not purpose-built for it: recalculating percentages by hand, forgetting who scored what, and comparing students against the class average all become tedious as the class grows. There is a need for a lightweight, dependency-free tool that a student or teacher can run immediately - without setting up a database or web server - to manage records and generate performance insights on demand.

Target Users

- Students who want a personal tool to track their own subject-wise performance across a semester.
- Teachers / class coordinators managing a small class or lab batch who need quick add/update/search and instant class statistics.

3. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output structure:

3.1 Student Management

- Add Student — input: name; output: system-assigned unique ID
- Update Student — input: ID + new name; output: confirmation or 'not found' error
- Delete Student — input: ID; output: confirmation or 'not found' error
- Search Student — input: ID or name fragment; output: matching student record(s)

3.2 Marks Management

- Enter Marks — input: student ID, subject name, marks (0-100); output: confirmation, rejected if out of range
- Calculate Percentage — derived automatically from all recorded subjects for a student
- Calculate Grade — derived automatically from percentage using a fixed grading scale

3.3 Performance Reports

- Individual Report — full report card for one student (subjects, total, percentage, grade, pass/fail)
- Class Average — mean percentage across every student currently in the system
- Highest / Lowest — identifies the top and bottom performer by percentage
- Pass/Fail Analysis — counts and lists students below the 40% pass threshold, and reports the class pass rate

Logical workflow: the user is shown a numbered menu, picks an option, supplies the requested input, and immediately sees the result before returning to the menu - this loop continues until the user chooses to exit, at which point all data is saved to disk.

4. Non-Functional Requirements

Requirement	How it is addressed
Usability	Simple numbered menu, one action at a time, clear prompts and confirmation/error messages after every action.
Reliability	All input is validated before being stored; the program never crashes on bad input - it catches the problem and lets the user retry.
Maintainability	Layered design (model / manager / report / persistence / UI) means each concern lives in its own class and can be changed independently.
Error Handling Strategy	Two custom checked exceptions (InvalidMarksException, StudentNotFoundException) force every risky operation to be explicitly handled instead of failing silently.
Performance	All operations are in-memory list/map lookups ($O(n)$ at worst for a class-sized roster), so every menu action responds instantly.
Resource Efficiency	No external database or framework; data is stored in a small plain-text CSV file, so memory and disk footprint stay minimal.

5. System Architecture

The application follows a simple three-layer architecture. The Presentation Layer (Main.java) only handles the console menu and input; the Business Logic Layer (StudentManager, Student, PerformanceReport, and the two custom exceptions) contains all the actual rules and calculations; and the Data Persistence Layer (FileManager + students.csv) is solely responsible for saving and restoring state. Each layer only calls into the layer directly below it, which keeps responsibilities cleanly separated.

System Architecture Diagram

Student Performance Management System

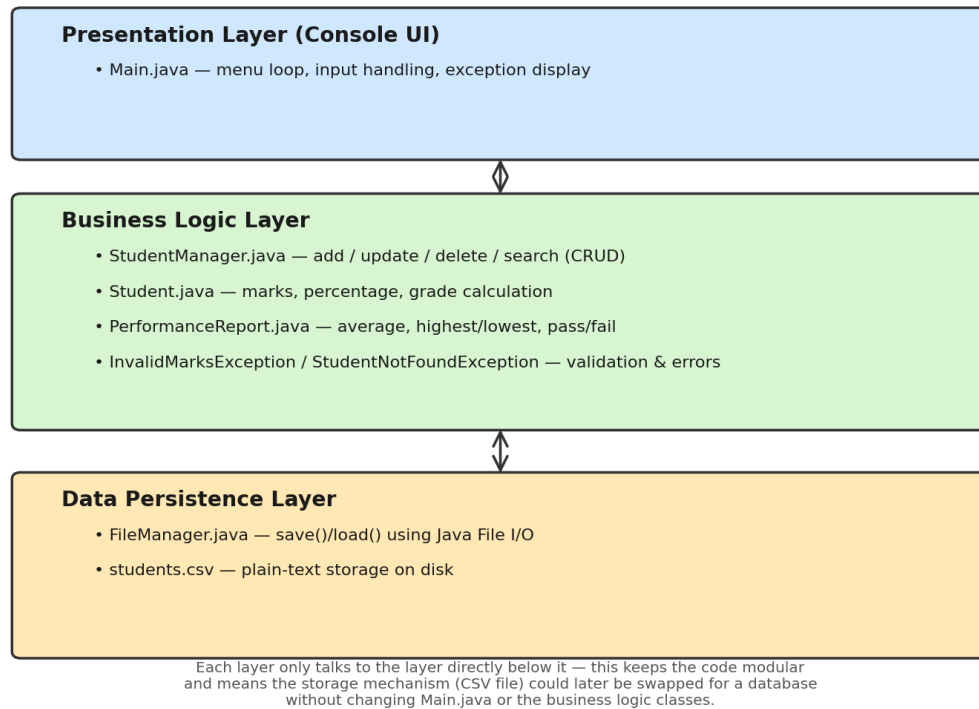


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

Describes the full lifecycle of one program run: load any previously saved data, loop over the menu until the user exits, and persist data back to disk both after every change and on exit.

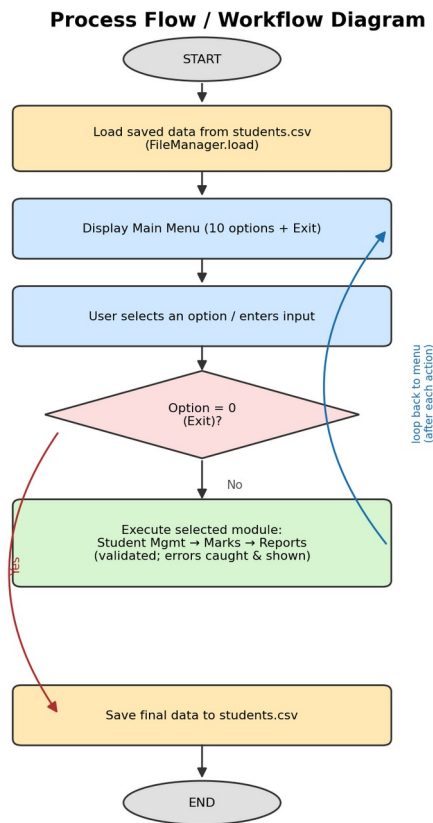


Figure 2: Process Flow / Workflow Diagram

6.2 Use Case Diagram

A single actor (Student or Teacher) drives all ten use cases, grouped under the three functional modules.

Use Case Diagram

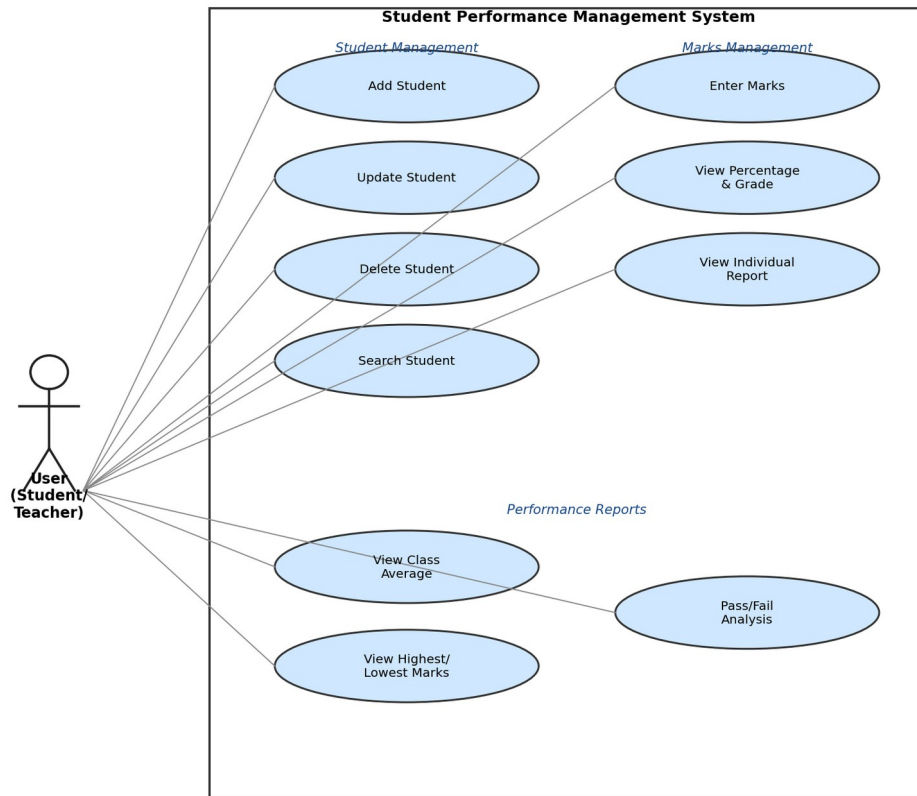


Figure 3: Use Case Diagram

6.3 UML Class Diagram

Shows every class in the codebase, their key fields/methods, and how they relate: StudentManager owns a collection of Student objects, Main coordinates the manager/report/file classes, and both Student and StudentManager can throw the two custom exceptions.

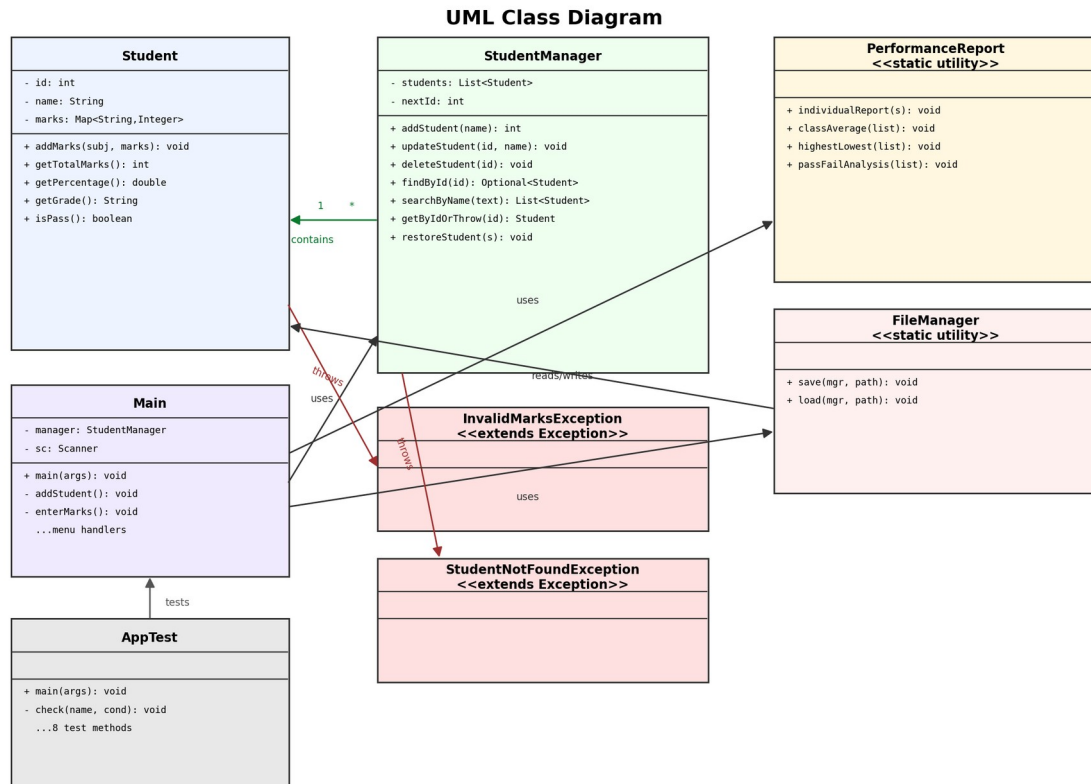


Figure 4: UML Class Diagram

6.4 Sequence Diagram

Traces a single representative flow - 'Enter Marks' - end to end: the console reads input, StudentManager looks up the student (or throws), Student validates and stores the marks (or throws), and FileManager persists the updated roster before Main reports back to the user.

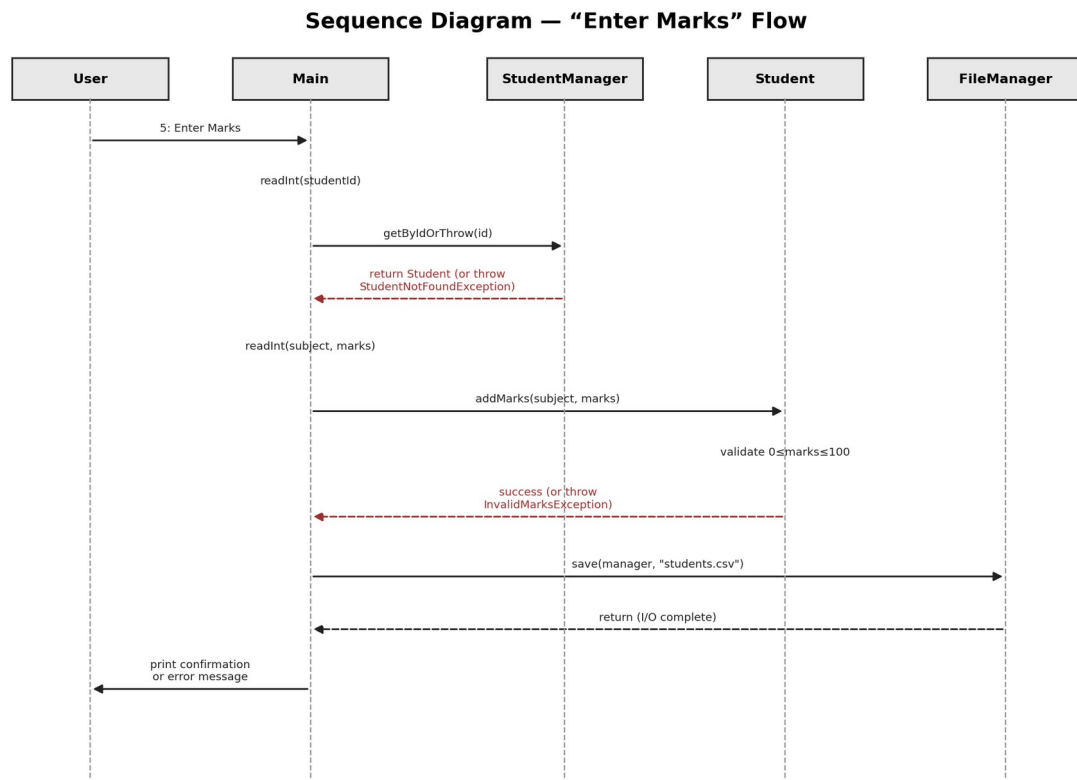


Figure 5: Sequence Diagram — Enter Marks Flow

6.5 Database / Storage Design

This project does not use a relational database, so no ER diagram applies. Storage is a flat CSV file, one line per student, in the format:

```
id,name,subject1:marks1;subject2:marks2;...
```

Example: 1,Piyush,Maths:90;Science:80

FileManager.java is the only class that reads or writes this format, so switching to a real database later would only require rewriting that one class.

7. Design Decisions & Rationale

- Layered design over one big class: splitting Student / StudentManager / PerformanceReport / FileManager / Main means each class has one reason to change, and the logic is easy to unit-test in isolation (see AppTest.java).
- Map<String,Integer> for marks instead of fixed fields: lets a student have any number of subjects without hard-coding 'subject1, subject2, ...' fields.
- Custom checked exceptions instead of boolean return values: forces the caller (Main) to explicitly decide what happens on bad input rather than silently ignoring a 'false' return, which is easy to forget.
- Plain CSV file instead of a database: the assignment's scope (a single class/section, run locally) does not justify the added complexity of a DB engine or ORM; CSV is human-readable and trivial to inspect while debugging.
- Auto-save after every mutating action: guarantees the file on disk is never more than one action out of date, protecting against data loss if the program is closed unexpectedly.

8. Implementation Details

The project has 8 Java source files under src/:

File	Responsibility
Student.java	Model: one student's ID, name, marks; calculates total, percentage, grade
StudentManager.java	Owns the ArrayList<Student> roster; add/update/delete/search
PerformanceReport.java	Static report methods: individual report, class average, highest/lowest, pass/fail
FileManager.java	Saves/loads the roster to/from students.csv using Java File I/O
InvalidMarksException.java	Custom checked exception for out-of-range or blank marks input
StudentNotFoundException.java	Custom checked exception for lookups on a non-existent ID
Main.java	Console menu loop; wires the other classes together; central exception handling
AppTest.java	Dependency-free test harness covering all core logic

Key Algorithms / Logic

- Grading scale: ≥ 90 A+, ≥ 80 A, ≥ 70 B, ≥ 60 C, ≥ 50 D, ≥ 40 E, else F
- Pass/Fail rule: a student passes if overall percentage $\geq 40\%$
- Highest/Lowest: single linear scan over the roster comparing `getPercentage()`
- ID assignment: an auto-incrementing counter in `StudentManager`, kept ahead of any ID restored from disk so new and reloaded IDs never collide

Validation & Error Handling

`Student.addMarks()` rejects marks outside 0-100 and blank subject names by throwing `InvalidMarksException`. `StudentManager.getIdOrThrow()` throws `StudentNotFoundException` for any ID that doesn't exist. `Main.java` catches both exception types in one place around the menu dispatch, so every operation fails safely with a clear message instead of crashing or corrupting the in-memory roster.

9. Screenshots / Results

The following screenshots show an actual program run: adding two students, entering marks, and generating each type of report.

```
java -cp bin Main

===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

Enter your choice: [USER] 7

Enter student ID: [USER] 1

----- Individual Report -----

ID   : 1

Name : Piyush

Subject-wise marks:

    Maths      : 90/100
    Science    : 80/100

Total       : 170 / 200

Percentage  : 85.00%

Grade      : A

Result     : PASS

-----
```

Figure 6: Individual Report output

```
java -cp bin Main

Enter your choice: [USER] 8

Class Average Percentage: 58.75% (based on 2 students)

Enter your choice: [USER] 9

Highest Scorer: ID: 1  Name: Piyush  Percentage: 85.00%  Grade: A
Lowest Scorer : ID: 2  Name: Ankit   Percentage: 32.50%  Grade: F

Enter your choice: [USER] 10

----- Pass/Fail Analysis -----

FAILED: ID: 2  Name: Ankit  Percentage: 32.50%  Grade: F

Total Students : 2

Passed      : 1
Failed      : 1
Pass Rate   : 50.00%

Enter your choice: [USER] 5

Enter student ID: [USER] 1

Enter subject name: [USER] Maths

Enter marks obtained (0-100): [USER] 150

Error: Marks must be between 0 and 100. Got: 150
```

Figure 7: Class Average, Highest/Lowest, Pass/Fail Analysis, and a rejected invalid-marks entry

10. Testing Approach

Testing is done with a small, dependency-free harness (AppTest.java) rather than an external framework like JUnit, since the assignment scope did not call for build-tool dependencies. Each test method exercises one behaviour and prints a PASS/FAIL verdict; a summary line reports the totals.

Test	What it verifies
testAddAndFindStudent	New students get sequential IDs and can be found by ID
testUpdateStudent	Updating a name persists on the same object
testDeleteStudent	Deleted students can no longer be found
testUpdateNonExistentStudentThrows	StudentNotFoundException is thrown for a bad ID
testGradeCalculation	Percentage and grade are computed correctly from marks
testInvalidMarksRejected	Marks above 100 or negative are rejected with InvalidMarksException
testClassAverageAndPassFail	Class average and individual pass/fail results are correct
testFilePersistence	A save() followed by load() round-trips a student's data exactly

Result on the current codebase: all 16 assertions pass (java -cp bin AppTest).

11. Challenges Faced

- Deciding between returning boolean flags versus throwing custom exceptions for failure cases - exceptions were chosen because they cannot be silently ignored the way an unchecked boolean return can.
- Keeping the auto-incrementing ID counter consistent after reloading data from disk, so a freshly added student can never accidentally reuse an ID that was restored from a previous session.
- Designing the CSV format to support a variable number of subjects per student without needing a more complex format - solved with a nested delimiter scheme (comma for fields, semicolon between subjects, colon between subject and mark).

12. Learnings & Key Takeaways

- Custom checked exceptions make error paths visible in the code (via throws clauses) instead of hidden inside 'if (!ok)' checks scattered through the caller.
- Separating a program into model / manager / report / persistence / UI layers makes each piece independently testable, which is what made the AppTest harness straightforward to write.
- A simple plain-text file format, designed deliberately, can support fairly rich data (a variable number of subjects per student) without needing a database.

13. Future Enhancements

- Support multiple classes/sections instead of a single flat roster
- Replace the CSV file with a real database (e.g. SQLite via JDBC) for larger datasets
- Add a graphical (JavaFX/Swing) or web-based front end in place of the console menu
- Add authentication so students and teachers see different views/permissions
- Export individual and class reports to PDF/Excel

14. References

- Oracle Java SE 17 Documentation — docs.oracle.com/en/java/javase/17/
- Java Collections Framework guide — docs.oracle.com/javase/tutorial/collections/
- Java Exceptions tutorial — docs.oracle.com/javase/tutorial/essential/exceptions/

- VITyarthi 'Build Your Own Project' guidelines document (course-provided)